
LAMP 環境構築課題

作成日：2026 年 6 月 24 日

作成者：高田晴斗

対象環境：Windows 11 + VirtualBox + AlmaLinux 9.8

1. はじめに

本課題では、LAMP 環境を構築し、Apache、PHP、MySQL、WordPress が連携して動作する環境を作成した。

構築した環境では、以下を確認した。

- Apache の VirtualHost を利用し、2 つのホスト名で異なる Web ページを表示できること
 - PHP 7 系をインストールし、phpinfo() をブラウザで表示できること
 - MySQL 8 系をインストールし、PHP から MySQL のデータを取得して表示できること
 - WordPress を構築し、ブログ記事を投稿できること
-

2. 使用環境

項目	内容
ホスト OS	Windows 11
仮想化ソフト	Oracle VM VirtualBox
ゲスト OS	AlmaLinux 9.8
Web サーバ	Apache httpd 2.4.62
PHP	PHP 7.4.33
DB	MySQL 8.0.46
CMS	WordPress
名字側ホスト名	takada-hbtask.local
名前側ホスト名	haruto-hbtask.local

3. OS・仮想環境の準備

3.1 VirtualBox の利用

Windows 11 上に Linux サーバ環境を構築するため、VirtualBox を利用した。

VirtualBox を選択した理由は、自分の PC 上に独立した Linux 環境を作成でき、OS イン

ストール、ネットワーク設定、Web サーバ構築などを実際のサーバに近い形で学習できるためである。

参考：VirtualBox 公式ダウンロードページでは、Windows hosts 向けのインストーラーが提供されている。

3.2 AlmaLinux の選択理由

Linux ディストリビューションには AlmaLinux 9.8 を選択した。

理由は、AlmaLinux が RHEL 互換の Linux ディストリビューションであり、今回の課題で扱う以下の要素を学びやすいと考えたためである。

- dnf
- firewalld
- SELinux
- httpd
- systemctl

AlmaLinux 公式のインストールガイドでは、AlmaLinux 9 系の ISO として boot、minimal、dvd などの種類が説明されている。今回は必要なパッケージを後から追加して学習しやすくするため、Minimal Install を選択した。

3.3 OS 確認

AlmaLinux インストール後、以下のコマンドで OS 情報を確認した。

```
cat /etc/os-release
```

確認結果：

```
NAME="AlmaLinux"
```

```
VERSION="9.8 (Olive Jaguar)"
```

```
ID="almalinux"
```

```
VERSION_ID="9.8"
```

```
PLATFORM_ID="platform:el9"
```

また、カーネル情報を以下で確認した。

```
uname -a
```

3.4 失敗したこと：コマンドのスペース忘れ

最初に OS 情報を確認する際、以下のように誤って入力した。

```
cat/haruto/os-release
```

その結果、以下のようなエラーが発生した。

```
No such file or directory
```

原因は、cat コマンドとファイルパスの間にスペースが必要であることを理解していなか

ったためである。

正しくは以下である。

```
cat /etc/os-release
```

この経験から、Linux コマンドは基本的に以下の構造であることを理解した。

コマンド オプション 引数

4. 作業ログの取得

課題では、実施した作業や失敗内容も記録する必要があるため、script コマンドを利用して作業ログを取得した。

```
script -a /home/haruto/hbtask_work.log
```

script コマンドにより、以降のターミナル操作をログファイルに追記できる。

4.1 失敗したこと：\$home と \$HOME の違い

最初に以下のように入力した。

```
script -a $home/hbtask_work.log
```

その結果、以下のようなエラーが発生した。

```
script: cannot open /hbtask_work.log: Permission denied
```

原因は、Linux の環境変数は大文字・小文字を区別するため、\$home ではなく \$HOME と書く必要があったためである。

\$home は空として扱われ、結果として /hbtask_work.log に保存しようとしていた。一般ユーザーは / 直下に書き込み権限がないため、Permission denied になった。

そのため、最終的には以下のようにフルパスで指定した。

```
script -a /home/haruto/hbtask_work.log
```

5. タイムゾーン設定

5.1 現在のタイムゾーン確認

```
timedatectl
```

5.2 JST への変更

```
sudo timedatectl set-timezone Asia/Tokyo
```

5.3 変更後の確認

```
timedatectl
```

確認結果として、以下が表示された。

```
Time zone: Asia/Tokyo (JST, +0900)
```

5.4 理解したこと

timedatectl は、systemd 環境で日時、タイムゾーン、NTP 設定などを確認・変更するため

のコマンドである。

サーバではログの時刻、DB の記録時刻、証明書の有効期限などが重要になるため、正しいタイムゾーン設定が必要である。

5.5 失敗したこと：コマンド名のスペルミス

一度、以下のように誤入力した。

```
timedetactl
```

結果として以下のエラーが出た。

```
command not found
```

正しくは以下である。

```
timedatectl
```

コマンド名を正確に入力する必要があることを理解した。

6. chrony による時刻同期

6.1 chrony のインストール

```
sudo dnf install -y chrony
```

6.2 設定ファイルのバックアップ

```
sudo cp -p /etc/chrony.conf /etc/chrony.conf.bak
```

設定変更前にバックアップを作成した。

これは、設定を誤った場合に元に戻せるようにするためである。

6.3 chrony 設定ファイルの編集

```
sudo nano /etc/chrony.conf
```

既存の NTP サーバ設定として、以下のような行があった。

```
server time-e-wwv.nist.gov iburst
```

課題では以下の NTP サーバとの同期が指定されていたため、設定ファイルに以下を追加した。

```
server ntp1.jst.mfeed.ad.jp iburst
```

```
server ntp2.jst.mfeed.ad.jp iburst
```

```
server ntp3.jst.mfeed.ad.jp iburst
```

6.4 chronyd の再起動と自動起動

```
sudo systemctl restart chronyd
```

```
sudo systemctl enable chronyd
```

6.5 起動確認

```
systemctl status chronyd
```

確認結果：

Active: active (running)

6.6 同期先確認

```
chronyc sources -v
```

確認結果として、以下の NTP サーバが表示された。

ntp1.jst.mfeed.ad.jp

ntp2.jst.mfeed.ad.jp

ntp3.jst.mfeed.ad.jp

6.7 理解したこと

chrony は NTP による時刻同期を行うためのソフトウェアである。

サーバでは、ログ調査、DB の記録、アプリケーションの時刻処理などで正確な時刻が必要になる。そのため、NTP サーバと同期することが重要である。

7. firewalld の有効化

7.1 firewalld の起動・自動起動

```
sudo systemctl enable --now firewalld
```

7.2 状態確認

```
systemctl status firewalld
```

確認結果：

Active: active (running)

7.3 HTTP 通信の許可

Apache で Web ページを表示するため、HTTP 通信を許可した。

```
sudo firewall-cmd --add-service=http --permanent
```

```
sudo firewall-cmd --reload
```

```
sudo firewall-cmd --list-all
```

確認結果として、services に http が含まれていることを確認した。

```
services: cockpit dhcpv6-client http ssh
```

7.4 理解したこと

firewalld は Linux のファイアウォール管理サービスである。

Apache で Web ページを公開するためには、外部からの HTTP 通信を許可する必要がある。

--permanent を付けることで再起動後も設定が残り、--reload により永続設定を現在の実行中設定へ反映できる。

7.5 失敗したこと：systemctl の操作名不足

最初に以下のように入力した。

```
sudo systemctl --now firewalld
```

しかし、正しくは以下である。

```
sudo systemctl enable --now firewalld
```

systemctl では、start、enable、restart、status などの操作名を指定する必要があることを理解した。

8. SELinux の無効化

8.1 状態確認

```
getenforce
```

初期状態では以下のように表示された。

```
Enforcing
```

8.2 一時的に Permissive へ変更

```
sudo setenforce 0
```

8.3 設定ファイルのバックアップ

```
sudo cp -p /etc/selinux/config /etc/selinux/config.bak
```

8.4 設定ファイルの編集

```
sudo nano /etc/selinux/config
```

以下の行を変更した。

変更前：

```
SELINUX=enforcing
```

変更後：

```
SELINUX=disabled
```

8.5 再起動

```
sudo reboot
```

8.6 再起動後の確認

```
getenforce
```

確認結果：

```
Disabled
```

8.7 理解したこと

SELinux は Linux の強制アクセス制御機能である。

本来はセキュリティを高めるために有効化して運用すべき機能だが、今回の課題要件では無効化が指定されていたため、設定ファイルを変更して無効化した。

9. Apache のインストールと起動

Apache HTTP Server の公式ドキュメントでは、VirtualHost により 1 台のサーバで複数のホスト名に対して異なる Web サイトを提供できることが説明されている。

9.1 Apache のインストール

```
sudo dnf install -y httpd
```

9.2 バージョン確認

```
httpd -v
```

確認結果：

Server version: Apache/2.4.62 (AlmaLinux)

Apache 2.4 系がインストールされていることを確認した。

9.3 起動・自動起動

```
sudo systemctl enable --now httpd
```

9.4 状態確認

```
systemctl status httpd
```

確認結果：

Active: active (running)

9.5 警告について

`httpd -t` や `systemctl status httpd` の確認時に、以下の警告が表示された。

AH00558: httpd: Could not reliably determine the server's fully qualified domain name

これは、Apache がサーバ自身の FQDN を判断できないという警告である。

ただし、Syntax OK が表示され、Apache 自体も active (running) だったため、Web サーバの動作には問題ないと判断した。

10. Apache VirtualHost 設定

10.1 DocumentRoot 用ディレクトリ作成

```
sudo mkdir -p /var/www/takada-hbtask.local
```

```
sudo mkdir -p /var/www/haruto-hbtask.local
```

10.2 表示用 HTML 作成

日本語入力が難しかったため、HTML 文字参照を使って HTML ファイルを作成した。

ブラウザでは日本語として表示される。

名字側：

```
echo
```

```
'&#12371;&#12435;&#12395;&#12385;&#12399;&#12290;&#12354;&#12426;&#12364;&#12392;&#12358;&#12290;' | sudo tee /var/www/takada-hbtask.local/index.html
```

名前側：

echo

```
'&#12354;&#12426;&#12364;&#12392;&#12358;&#12290;&#12371;&#12435;&#12395;&#12385;&#12399;&#12290;' | sudo tee /var/www/haruto-hbtask.local/index.html
```

ブラウザ表示結果：

takada-hbtask.local

こんにちは。ありがとう。

haruto-hbtask.local

ありがとう。こんにちは。

10.3 VirtualHost 設定ファイル作成

```
sudo tee /etc/httpd/conf.d/hbtask-vhosts.conf > /dev/null <<'EOF'
```

```
<VirtualHost *:80>
```

```
    ServerName takada-hbtask.local
```

```
    DocumentRoot /var/www/takada-hbtask.local
```

```
    <Directory "/var/www/takada-hbtask.local">
```

```
        Require all granted
```

```
    </Directory>
```

```
</VirtualHost>
```

```
<VirtualHost *:80>
```

```
    ServerName haruto-hbtask.local
```

```
    DocumentRoot /var/www/haruto-hbtask.local
```

```
    <Directory "/var/www/haruto-hbtask.local">
```

```
        Require all granted
```

```
    </Directory>
```

```
</VirtualHost>
```

```
EOF
```

10.4 Apache 設定確認

```
sudo httpd -t
```

確認結果：

Syntax OK

VirtualHost 確認：


```
sudo httpd -S
```

確認結果として、以下の VirtualHost が認識されていることを確認した。

```
takada-hbtask.local
```

```
haruto-hbtask.local
```

10.5 Apache 再起動

```
sudo systemctl restart httpd
```

10.6 Linux 内での表示確認

```
curl -H 'Host: takada-hbtask.local' http://localhost/
```

```
curl -H 'Host: haruto-hbtask.local' http://localhost/
```

それぞれ異なる内容が返ることを確認した。

10.7 Windows hosts ファイル設定

Windows 側の hosts ファイルに以下を追記した。

ファイルパス：

```
C:¥Windows¥System32¥drivers¥etc¥hosts
```

追記内容：

```
127.0.0.1 takada-hbtask.local
```

```
127.0.0.1 haruto-hbtask.local
```

Windows の hosts ファイルは、IP アドレスとホスト名を対応付けるためのプレーンテキストファイルであり、# で始まる行はコメントとして扱われる。

10.8 VirtualBox のポートフォワーディング

VirtualBox の NAT 環境を利用していたため、以下のポートフォワーディングを設定した。

SSH:

ホストポート 2222 → ゲストポート 22

HTTP:

ホストポート 80 → ゲストポート 80

これにより、Windows 側のブラウザから 127.0.0.1:80 経由で仮想マシン内の Apache へ接続できるようにした。

10.9 ブラウザでの確認

以下の URL にアクセスした。

```
http://takada-hbtask.local/
```

表示結果：

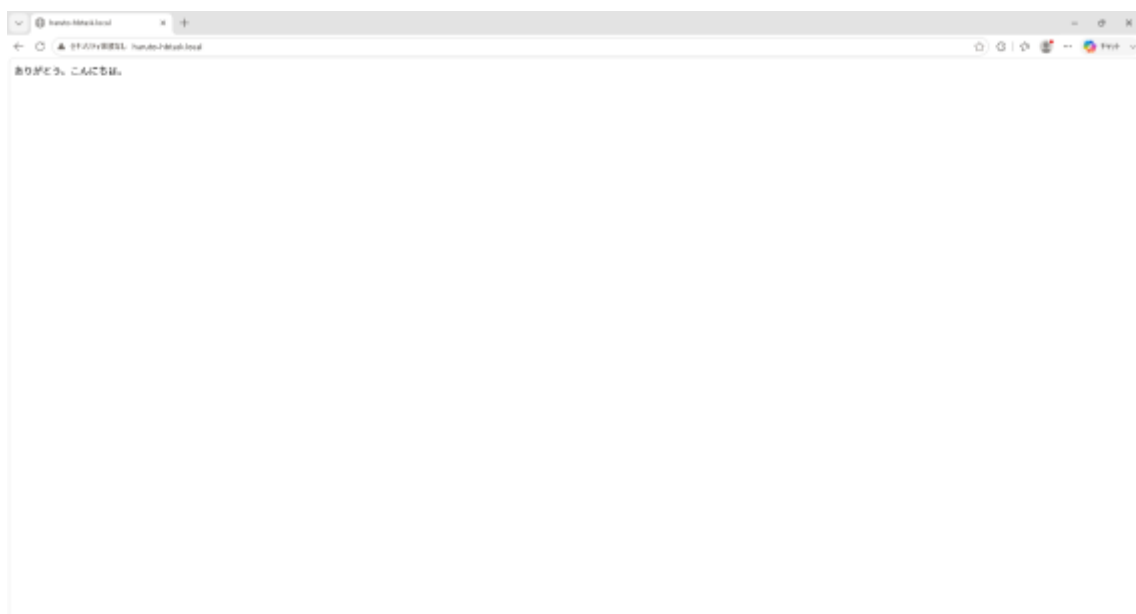
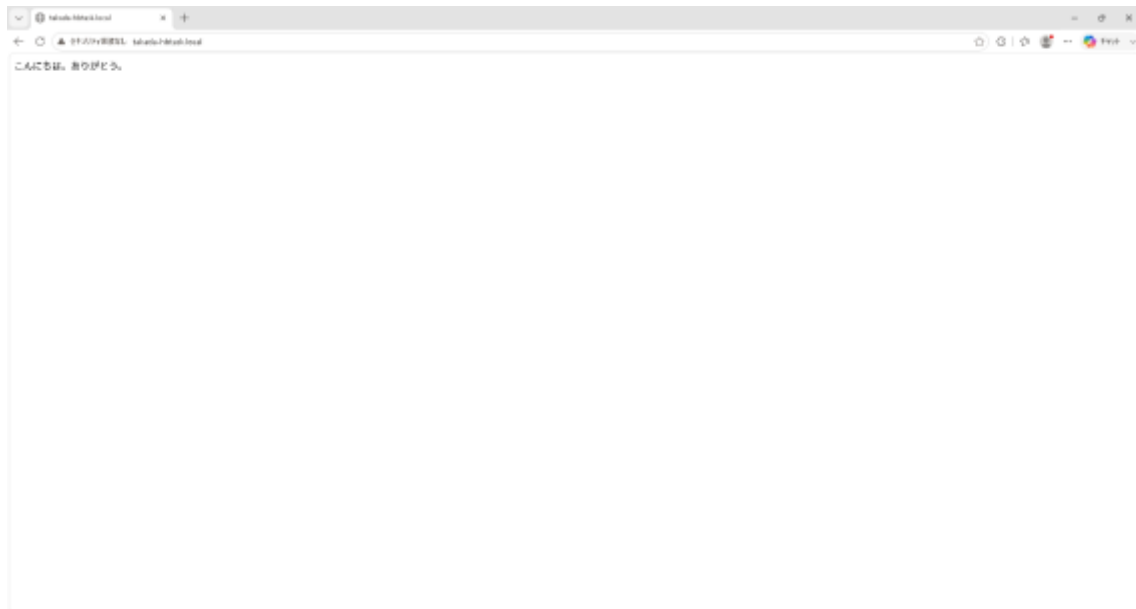
こんにちは。ありがとう。

以下の URL にもアクセスした。

<http://haruto-hbtask.local/>

表示結果：

ありがとう。こんにちは。



10.10 失敗したことと解決方法

mkdir の入力ミス

最初に以下のように誤入力した。

```
sudo mdir -p /var/www/haruto-hbtask.local
```

結果：

```
command not found
```

原因は、ディレクトリ作成コマンド `mkdir` を `mdir` と誤入力したためである。

正しくは以下である。

```
sudo mkdir -p /var/www/haruto-hbtask.local
```

apachectl -S が使えなかった

VirtualHost 設定確認のため、最初に以下を実行した。

```
sudo apachectl -S
```

しかし、以下のエラーが出た。

```
apachectl: The "-S" option is not supported.
```

そのため、以下を使用した。

```
sudo httpd -S
```

環境によって利用できる確認コマンドが異なることを理解した。

11. SSH 接続への切り替え

VirtualBox のコンソール画面ではコピー&ペーストがうまくできず、長いコマンドを手入力するのが困難だった。

そのため、SSH 接続に切り替えた。

11.1 sshd 起動

```
sudo systemctl enable --now sshd
```

11.2 Windows PowerShell から SSH 接続

VirtualBox の NAT ポートフォワーディングで、ホストポート 2222 をゲストポート 22 に転送した。

Windows PowerShell から以下で接続した。

```
ssh -p 2222 haruto@127.0.0.1
```

SSH 接続後は、Windows 側からコマンドをコピー&ペーストできるようになり、作業効率が大きく改善した。

11.3 失敗したこと：SSH ログイン時のパスワード

最初は SSH ログイン時にパスワードが分からずログインできなかった。

そのため、AlmaLinux 側で以下を実行し、haruto ユーザーのパスワードを設定した。

```
sudo passwd haruto
```

その後、PowerShell から SSH 接続できるようになった。

12. PHP 7.4 のインストール

PHP 7 系をインストールするため、Remi リポジトリを利用した。Remi リポジトリには Enterprise Linux 9 向けの PHP 7.4 パッケージが提供されている。

12.1 Remi リポジトリの追加

```
sudo dnf install -y epel-release
```

```
sudo dnf install -y https://rpms.remirepo.net/enterprise/remi-release-9.rpm
```

12.2 PHP モジュールの切り替え

```
sudo dnf module reset -y php
```

```
sudo dnf module enable -y php:remi-7.4
```

12.3 PHP 関連パッケージのインストール

```
sudo dnf install -y php php-cli php-common php-mysqlnd php-gd php-mbstring php-xml  
php-json
```

12.4 バージョン確認

```
php -v
```

確認結果：

PHP 7.4.33

12.5 phpinfo.php の作成

```
echo '<?php phpinfo(); ?>' | sudo tee /var/www/takada-hbtask.local/phpinfo.php
```

12.6 Apache 再起動

```
sudo systemctl restart httpd
```

12.7 ブラウザ確認

以下にアクセスした。

<http://takada-hbtask.local/phpinfo.php>

phpinfo 画面が表示され、PHP バージョンが 7.4.33 であることを確認した。


```
systemctl status mysqld
```

確認結果：

Active: active (running)

13.5 理解したこと

MySQL はリレーショナルデータベース管理システムであり、Web アプリケーションのデータを保存・検索するために利用される。

systemctl enable --now mysqld により、MySQL を現在起動し、OS 再起動後も自動起動するように設定した。

14. hbtask データベースと meibo テーブルの作成

14.1 MySQL へ接続

```
sudo mysql
```

14.2 データベース作成

```
CREATE DATABASE hbtask CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci;
```

14.3 DB ユーザー作成

```
CREATE USER 'hbuser'@'localhost' IDENTIFIED BY '*****';
```

```
GRANT SELECT ON hbtask.* TO 'hbuser'@'localhost';
```

```
FLUSH PRIVILEGES;
```

パスワードは提出時には伏せる。

14.4 テーブル作成

```
USE hbtask;
```

```
CREATE TABLE meibo (  
    number INT,  
    username VARCHAR(100)  
);
```

14.5 データ登録

```
INSERT INTO meibo (number, username)
```

```
VALUES (1, 'Haruto Takada');
```

14.6 SQL での確認

```
SELECT username FROM meibo WHERE number = 1;
```

確認結果：

Haruto Takada

14.7 理解したこと

CREATE DATABASE はデータベースを作成する SQL である。

CREATE TABLE はテーブルを作成する SQL であり、今回は以下のカラムを作成した。

- number : INT 型
- username : VARCHAR(100)型

INSERT INTO によりデータを登録し、SELECT により条件に一致するデータを取得した。

また、Web アプリケーションから DB へ接続する際に root ユーザーを直接使うと権限が強すぎるため、専用ユーザー hbuser を作成し、必要な権限のみを付与した。

15. PHP から MySQL のデータを表示

15.1 mysql.php の作成

```
sudo tee /var/www/haruto-hbtask.local/mysql.php > /dev/null <<'EOF'
```

```
<?php
```

```
$mysqli = new mysqli('localhost', 'hbuser', '*****', 'hbtask');
```

```
if ($mysqli->connect_error) {
```

```
    die('DB 接続失敗: ' . htmlspecialchars($mysqli->connect_error, ENT_QUOTES, 'UTF-8'));
```

```
}
```

```
$mysqli->set_charset('utf8mb4');
```

```
$stmt = $mysqli->prepare('SELECT username FROM meibo WHERE number = ?');
```

```
$number = 1;
```

```
$stmt->bind_param('i', $number);
```

```
$stmt->execute();
```

```
$stmt->bind_result($username);
```

```
$stmt->fetch();
```

```
echo '私の名前は ' . htmlspecialchars($username, ENT_QUOTES, 'UTF-8') . ' です。';
```

```
$stmt->close();
```

```
$mysqli->close();
```

```
?>
```

```
EOF
```

実際のファイルでは、***** の部分に DB ユーザーのパスワードを設定した。
提出時にはパスワードは伏せる。

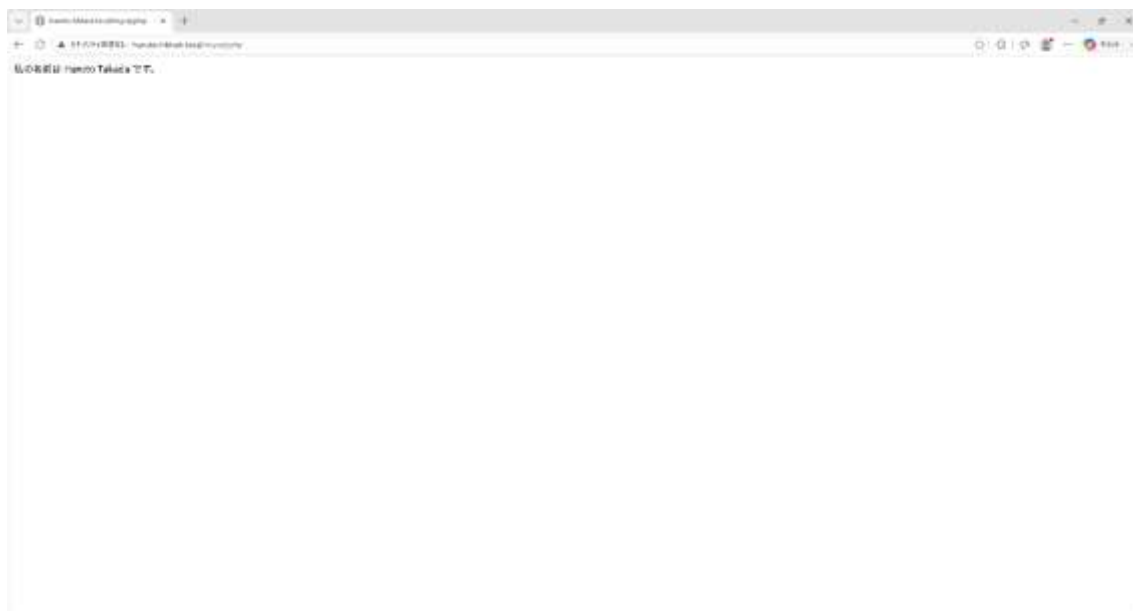
15.2 ブラウザ確認

以下にアクセスした。

`http://haruto-hbtask.local/mysql.php`

表示結果：

私の名前は Haruto Takada です。



15.3 失敗したこと：URL 末尾に不要文字が入った

最初にブラウザで確認した際、以下のような URL になっていた。

`http://haruto-hbtask.local/mysql.php%60%60`

その結果、Not Found が表示された。

原因は、コピー時にバッククォートが混ざり、URL エンコードされた %60%60 が末尾に付いてしまったためである。

正しい URL は以下である。

`http://haruto-hbtask.local/mysql.php`

正しい URL に修正したところ、期待通りの表示になった。

15.4 理解したこと

PHP の `mysqli` を利用することで、PHP から MySQL へ接続し、SQL を実行して結果をブラウザに表示できる。

今回は以下の SQL を実行した。


```
SELECT username FROM meibo WHERE number = 1
```

また、prepare と bind_param を利用することで、SQL 文と値を分離して扱える。これは SQL インジェクション対策の基本である。

htmlspecialchars は、DB から取得した文字列を HTML として出力する際に特殊文字をエスケープするために利用した。

16. WordPress の構築

WordPress 公式では、WordPress の動作環境として PHP、MySQL または MariaDB、Apache または Nginx などが説明されている。現在は PHP 8.3 以上が推奨されているが、今回の課題では PHP 7 系が指定されていたため、PHP 7.4 で構築した。

16.1 WordPress 用 DB 作成

```
sudo mysql
```

MySQL 内で以下を実行した。

```
CREATE DATABASE wordpress_hbtask CHARACTER SET utf8mb4 COLLATE  
utf8mb4_unicode_ci;
```

```
CREATE USER 'wpuser'@'localhost' IDENTIFIED BY '*****';
```

```
GRANT ALL PRIVILEGES ON wordpress_hbtask.* TO 'wpuser'@'localhost';
```

```
FLUSH PRIVILEGES;
```

パスワードは提出時には伏せる。

16.2 WordPress のダウンロード

```
cd /tmp
```

```
curl -O https://wordpress.org/latest.tar.gz
```

16.3 WordPress の展開

```
tar xzf latest.tar.gz
```

16.4 WordPress の配置

```
sudo mv wordpress /var/www/takada-hbtask.local/blog
```

16.5 所有者変更

```
sudo chown -R apache:apache /var/www/takada-hbtask.local/blog
```

16.6 Apache 再起動

```
sudo systemctl restart httpd
```

16.7 配置確認

```
ls -ld /var/www/takada-hbtask.local/blog
```

```
ls -l /var/www/takada-hbtask.local/blog/index.php
```

index.php が存在することを確認した。

16.8 ブラウザで初期設定

以下にアクセスした。

<http://takada-hbtask.local/blog/>

WordPress の初期設定画面が表示された。

入力した DB 情報は以下である。

データベース名：wordpress_hbtask

ユーザー名：wpuser

パスワード：*****

データベースのホスト名：localhost

テーブル接頭辞：wp_

16.9 サイト情報設定

WordPress の初期設定で以下を設定した。

サイト名：HB Task Blog

管理ユーザー名：admin-hbtask

パスワード：*****

パスワードは提出時には伏せる。

16.10 記事投稿確認

WordPress 管理画面から記事を 1 件投稿した。

投稿内容：

タイトル：LAMP 環境構築課題のテスト投稿

本文：

WordPress の投稿機能が利用できることを確認するためのテスト投稿です。

ブラウザで投稿ページを確認した。

<http://takada-hbtask.local/blog/?p=5>

表示結果として、投稿タイトルと本文が表示された。

これにより、WordPress のブログサイトが構築され、記事投稿ができることを確認した。



16.11 失敗したこと：WordPress で Not Found が表示された

最初に以下へアクセスした際、Not Found が表示された。

`http://takada-hbtask.local/blog/`

原因は、WordPress 本体が `/var/www/takada-hbtask.local/blog` に正しく配置されていなかったためである。

ターミナルでは以下のようなエラーが出ていた。

```
mv: cannot stat 'wordpress': No such file or directory
```

```
chown: cannot access '/var/www/takada-hbtask.local/blog': No such file or directory
```

これは、`latest.tar.gz` の展開が正常にできておらず、`wordpress` ディレクトリが存在しなかったことが原因である。

そのため、`tar` パッケージをインストールした。

```
sudo dnf install -y tar
```

その後、再度展開した。

```
tar xzf latest.tar.gz
```

`wordpress` ディレクトリが作成されたことを確認してから、以下で正しい場所へ移動した。

```
sudo mv wordpress /var/www/takada-hbtask.local/blog
```

これにより、WordPress の初期設定画面が表示されるようになった。

16.12 理解したこと

WordPress は PHP で動作する CMS であり、記事や設定情報を MySQL に保存する。

今回の構成では、ブラウザからのリクエストは以下の流れで処理される。

Windows ブラウザ

↓

hosts ファイルによる名前解決

↓

VirtualBox NAT ポートフォワーディング

↓

AlmaLinux 上の Apache

↓

PHP

↓

MySQL

WordPress 本体を Apache の DocumentRoot 配下に正しく配置しないと、Apache は該当 URL のファイルを見つけられず、Not Found を返すことを理解した。

17. 全体を通して学んだこと

今回の課題を通して、LAMP 環境の各要素がどのように連携しているかを理解できた。

特に、以下を学んだ。

17.1 Linux コマンドの基本

Linux では、コマンド、オプション、引数をスペースで区切る必要がある。

例：

```
cat /etc/os-release
```

この構造を理解していないと、意図しないコマンドとして解釈されることがある。

17.2 systemctl によるサービス管理

Apache、MySQL、chrony、firewalld などは、systemctl で起動・停止・自動起動設定を行う。

例：

```
sudo systemctl enable --now httpd
```

```
systemctl status httpd
```

enable は自動起動設定、--now は同時に現在起動する指定である。

17.3 設定変更前のバックアップの重要性

設定ファイルを変更する前に、以下のようにバックアップを作成した。

```
sudo cp -p /etc/chrony.conf /etc/chrony.conf.bak
```

```
sudo cp -p /etc/selinux/config /etc/selinux/config.bak
```

設定変更失敗した場合に戻せるようにするため、バックアップは重要である。

17.4 Apache VirtualHost の仕組み

Apache の VirtualHost では、HTTP リクエストの Host ヘッダーを見て、どの DocumentRoot を使うかを切り替える。

今回の場合、同じ Apache サーバでも以下のように異なる HTML を返すことができた。

takada-hbtask.local → /var/www/takada-hbtask.local

haruto-hbtask.local → /var/www/haruto-hbtask.local

17.5 hosts ファイルによる名前解決

Windows の hosts ファイルを編集することで、任意のホスト名を指定した IP アドレスに対応付けることができる。

今回は以下のように設定した。

127.0.0.1 takada-hbtask.local

127.0.0.1 haruto-hbtask.local

これにより、ブラウザから独自の .local ホスト名でアクセスできるようになった。

17.6 PHP と MySQL の連携

PHP から MySQL へ接続し、SQL で取得したデータをブラウザに表示できることを確認した。

今回の構成では、以下の流れで表示している。

ブラウザ

↓

Apache

↓

PHP mysql.php

↓

MySQL hbtask.meibo

↓

username 取得

↓

ブラウザへ表示

17.7 WordPress の仕組み

WordPress は、PHP と MySQL を利用して動作する CMS である。

投稿記事はファイルとして保存されるのではなく、MySQL 内に保存される。

ブラウザから WordPress へアクセスすると、PHP が MySQL から記事データを取得し、

HTML として表示する。

18. 今回発生した主なエラーと解決方法

発生した問題	原因	解決方法
cat/haruto/os-release が失敗	コマンドと引数の間にスペースがなかった	cat /etc/os-release に修正
timedetectl が失敗	コマンド名のスペルミス	timedatectl に修正
script -a \$home/... が失敗	\$home と \$HOME の違い	フルパス /home/haruto/... を使用
mkdir が失敗	mkdir の入力ミス	mkdir -p に修正
VirtualBox でコピペできない	CUI 環境では共有クリップボードが効きにくい	SSH 接続に切り替え
apachectl -S が失敗	環境上、-S がサポートされなかった	httpd -S を使用
URL 末尾に %60%60 が付いて 404	コピー時にバッククォートが混入	正しい URL に修正
VM 停止中に接続拒否	Apache が動く VM 自体が停止していた	VM を起動し直した
WordPress で Not Found	WordPress 本体が配置されていなかった	tar 展開後、正しい場所に配置

19. 参考 URL

• Oracle VM VirtualBox Downloads

<https://www.virtualbox.org/wiki/Downloads>

VirtualBox の Windows hosts 向けインストーラーを確認するために参照した。

• AlmaLinux Installation Guide

<https://wiki.almalinux.org/documentation/installation-guide.html>

AlmaLinux の ISO 種別、インストール方法、最小要件を確認するために参照した。

- Apache HTTP Server 2.4 VirtualHost Examples

<https://httpd.apache.org/docs/2.4/vhosts/examples.html>

Apache の VirtualHost 設定例を確認するために参照した。

- Apache Virtual Host documentation

<https://httpd.apache.org/docs/2.4/en/vhosts/>

VirtualHost の仕組みを確認するために参照した。

- Remi's RPM Repository

<https://rpms.remirepo.net/>

AlmaLinux 9 で PHP 7.4 をインストールするために参照した。

- MySQL 8.0 Reference Manual

https://docs.oracle.com/cd/E17952_01/mysql-8.0-en/

MySQL 8.0 の仕様、インストール、基本的な管理方法を確認するために参照した。

- WordPress Requirements

<https://wordpress.org/about/requirements/>

WordPress の動作要件を確認するために参照した。

- WordPress Download

<https://wordpress.org/download/>

WordPress 本体のダウンロード元として参照した。

- Microsoft Support: Hosts file

<https://support.microsoft.com/en-us/topic/how-to-reset-the-hosts-file-back-to-the-default-c2a43f9d-e176-c6f3-e4ef-3500277a6dae>

Windows の hosts ファイルの書式を確認するために参照した。

20. 次回試してみたいこと

今回の必須課題では、1 台のサーバ上に Apache、PHP、MySQL、WordPress を構築した。

次回は以下を試してみたい。

20.1 SELinux を有効化した状態での構築

今回は課題要件に従い SELinux を無効化したが、本番運用では SELinux を有効にしたまま運用することが望ましいと考えられる。

次回は SELinux を有効化した状態で、Apache や WordPress に必要なコンテキスト設定を調べながら構築してみたい。

20.2 HTTPS 化

今回は HTTP のみで構築した。

次回は自己署名証明書、または Let's Encrypt などを利用して HTTPS 化し、Web 通信を暗号化する構成を試してみたい。

20.3 Web サーバと DB サーバの分離

今回は 1 台の仮想マシンに Apache、PHP、MySQL をすべて入れた。

次回は Web サーバと DB サーバを分け、PHP から別サーバの MySQL へ接続する構成を試してみたい。

20.4 バックアップと復元

WordPress のファイルと MySQL データベースをバックアップし、別環境で復元できるかを試してみたい。

21. まとめ

今回の課題では、AlmaLinux 上に LAMP 環境を構築し、Apache、PHP、MySQL、WordPress が連携して動作することを確認した。

最初は Linux コマンドの入力ミスや VirtualBox 環境での操作に苦戦したが、エラー内容を確認しながら原因を切り分け、修正することで最終的にすべての必須要件を達成できた。特に、単にコマンドを実行するだけでなく、以下を理解できたことが大きな学びである。

- Linux サービスの起動・自動起動の仕組み
- firewalld による通信許可
- SELinux の役割
- Apache VirtualHost の仕組み
- hosts ファイルによる名前解決
- PHP と MySQL の連携
- WordPress が PHP と MySQL を利用して動作する仕組み

以上により、LAMP 環境の基本構成と、Web アプリケーションがブラウザに表示されるまでの流れを理解できた。